ProbOS — Month 1, Week 2
Model ABC + BatteryModel2Cell

Nisong Monyimba

July 3, 2026

Week 2 goal

Build the `Model` abstract base class defining the `forward_batch()` contract every domain model must satisfy, and the first concrete model: `BatteryModel2Cell`, an 8-state Arrhenius thermal-runaway ODE validated against Kim et al. (2007).

Day-by-day plan

Day 1 – Model ABC design

- Define the interface: `state_dim`, `param_dim`, `param_names()`, `initial_state()`, `forward_batch(state, params, dt)`
- Key constraint: `forward_batch` must be vectorised over N particles via NumPy broadcasting, no per-particle Python loops
- `python/src/state.py` skeleton

Day 2 – BatteryModel2Cell state and parameters

- Define the 8-state vector: two cells $\times$ (temperature, SEI concentration, anode concentration, cathode concentration)
- Define the 15-parameter vector: activation energies, pre-exponential factors, heat capacities per reaction
- `python/src/battery_model.py` skeleton

Day 3 – Arrhenius kinetics implementation

- Implement the three parallel exothermic reactions (SEI decomposition, anode reaction, cathode reaction) via the Arrhenius rate law
- Explicit Euler integration of the coupled ODE system
- Vectorise fully across particles

Day 4 – Parameter priors

- `python/src/parameter_priors.py`: 15 prior distributions built from the Week 1 `Distribution` types, centred on Kim et al. (2007) nominal values
- `build_battery_priors()` factory function

Day 5 – Kim 2007 validation

- Deterministic run at nominal parameters, adiabatic conditions
- Compare onset temperature and thermal runaway timing against Kim et al. (2007) accelerating rate calorimetry (ARC) data
- `python/examples/week2_battery_ode.py` deterministic demo script

Day 6 – CLT convergence demo

- `python/examples/week2_clt_demo.py`: verify Monte Carlo error shrinks at the theoretical $1/\sqrt{N}$ rate on a simple test case, ahead of building the full engine in Week 3

Day 7 – Test coverage + hardening

- `python/tests/test_state.py` – Model ABC contract tests
- `python/tests/test_battery_model.py` – 8-state ODE correctness, Kim 2007 validation as an automated test, physical invariants (temperatures stay positive, concentrations stay in $[0, 1]$)
- mypy strict, ruff clean, CI green

What was actually accomplished (retrospective)

- `Model` ABC implemented exactly as planned
- `BatteryModel2Cell`: 8-state Arrhenius thermal-runaway ODE, fully vectorised `forward_batch()`
- 15 prior distributions in `parameter_priors.py`, built entirely from Week 1's `Distribution` types – no new distribution code needed
- Kim et al. (2007) validation: onset temperature and runaway behaviour match the reference ARC data
- Deterministic ODE demo and CLT convergence demo scripts
- 67 new Python tests (test_state.py + test_battery_model.py)

Numbers at Week 2 close

- Python tests: 111/111 passing (44 Week 1 + 67 new)
- C++ tests: 13/13 passing (unchanged from Week 1)
- mypy strict: 0 errors
- ruff: 0 warnings

Carried into Week 3

`BatteryModel2Cell` + `build_battery_priors()` become the standard test case for every subsequent kernel layer: the Monte Carlo engine, Sobol sensitivity analysis, and provenance tracking in Week 3 are all built and validated against this exact model.